DOM Clobbering

- 前期工作
  - 研究背景
    - 早期Web安全：向易受攻击页面注入脚本代码
    - 当今Web安全：通过注入看似无害的HTML标记利用交互劫持程序执行
    - DOM劫持
      - 原理：通过在 HTML 中注入带有 id 或 name 属性的标签，导致浏览器在执行 JavaScript 时，将这些 HTML 元素误认为全局变量或对象的属性。
      - 后果：破坏脚本逻辑、绕过安全检查或实现 XSS
  - 前期基础
    - 此前研究主要集中在Script Gadgets
    - 涉及标签、属性、代码特性、运行行为的组合攻击，仅触及表面，缺乏系统性和全面性
  - 作者看法
    - 防范DOM胁持攻击主要由开发者负责，但现有可使用的防护措施存在不完整甚至不足的问题
    - 开发人员对DOM劫持漏洞所带来的风险并未重视
- 具体研究点
  - 对于该漏洞的首次系统性和全面性研究
    - 对攻击面的系统性探索
    - 对受影响及易受攻击网站的实证测量
    - 对防御措施的审查与评估
- 解决方案
  - TheThing
    - DOM劫持检测工具
      - 静态分析
        - 使用Puppeteer和 CDP（ Chrome DevTools Protocol），爬取网页，收集所有的 JavaScript 脚本和运行时的 DOM 状态。
        - 将代码转化为图结构并进行污点追踪，查找是否存在从源（被覆盖的变量）到汇（敏感操作）的数据流
      - 动态验证
        - 强制执行
          - 利用生成的DOM劫持标记进行自动化漏洞验证
  - 防御与评估
    - 现有防御机制的失效分析
      - 浏览器级防御
        - 禁用命名属性访问：虽然可以通过关闭“通过 id 访问元素”的功能来根除该漏洞，但这会造成巨大的兼容性问题。
      - HTML清洗库
        - 清洗不彻底：55%库在默认配置下是无效的，仍受漏洞影响
      - 内容安全策略
        - CSP 可以防止 XSS，但对 DOM Clobbering 作用有限。
    - 提出安全编码模式
      - 显式变量声明
        - 始终使用 var, let 或 const
        - 确保变量存在于 JavaScript 自己的作用域内
      - 严格类型检查
        - 在使用变量前进行 instanceof 或 typeof 检查
        - 确保变量是一个预期的对象/字符串，而不是一个被攻击者注入的 HTML 元素
      - 命名空间隔离
        - 对用户生成的 HTML 内容进行 id/name 前缀化处理
      - 使用 document 获取全局变量
        - 开发者应习惯使用 globalThis 或局部作用域
- 方案评价
  - 知识系统化的卓越贡献
    - 将一个“碎片化”的漏洞提升到了“系统化”的高度
    - 通过穷举式生成 3100 多万种标记组合，定义了4种核心技术规则
  - 技术方案的逻辑闭环
    - 静态分析+动态验证，即保证大规模扫描的效率，又降低了误报率

Presented with xmind